

EXPLAINER GUIDE

Understanding Project LOOP

A plain-English walkthrough of the Zidio Internship Project Brief — what you're building, why, and how to get through all 4 weeks.

Item	Detail
Source document	Zidio Project_Web_1_1.pdf (Project LOOP brief, v1.0)
Purpose of this guide	Explain the project in plain language so you can start building with confidence
Best used	Alongside the original brief — this is a companion, not a replacement

How to use this guide: Read straight through once to get the big picture, then come back to individual sections week by week as you build.

1. What Are You Actually Building?

Project LOOP is a **web app that reads customer feedback for a company and tells them what customers actually want**. Think of it like this: a company gets feedback from support tickets, app reviews, surveys, and sales calls — hundreds of scattered messages a week. Nobody has time to read them all. LOOP swallows all of that text, uses AI (Claude) to tag, sort, and summarize it, and turns it into a dashboard a product manager can actually use to decide what to build next.

It is **not** a to-do list app or a chatbot glued onto a blog. It's modeled on real SaaS products like Enterpret, Dovetail, or Productboard Insights — tools companies genuinely pay for.

The one sentence that matters

"LOOP turns scattered customer feedback into a ranked, evidence-backed list of what to do next."
Keep this in mind — your whole build and your demo should prove this sentence true.

The real-world problem it solves

Companies collect feedback everywhere — tickets, reviews, NPS surveys, sales notes, social posts — but each item alone is just a sentence. Nobody reads and tags hundreds of items a week, so the feedback sits ignored and decisions get made on gut feeling instead of evidence. LOOP fixes this by doing the reading, tagging, and pattern-spotting automatically.

2. Why This Project Was Chosen For You

- **Portfolio value** — "I built a multi-tenant AI feedback-intelligence platform" is a strong interview line, not a tutorial clone.
- **Hard to fake** — tenant isolation, role-based access, and AI grounded in real data can't be copy-pasted from a single tutorial video.
- **Measurable** — the required features map directly onto the grading rubric, so your score reflects what you actually shipped.

3. Core Concepts You Must Understand Before You Start

Multi-tenancy

"Tenant" = one company using LOOP (called a **Workspace** in the schema). Many companies share the same app and database, but each company's data must be completely invisible to every other company. This is the single most important technical rule in the whole project: **every database query that touches feedback, themes, reports, or users must filter by workspaceId**. Mentors will test this directly — for example, trying to view another company's feedback just by guessing an ID in the URL.

Role-Based Access Control (RBAC)

Three roles exist inside each workspace:

Role	What they can do
Admin	Everything — manage members, assign roles, full data access
Analyst	Ingest and manage feedback, use AI features, cannot manage members

Role	What they can do
Viewer	Read-only access to dashboards and reports

Roles must be enforced in your **API code**, not just hidden in the UI — attempting a forbidden action should return an HTTP 403 error, not a crash or silent failure.

Retrieval-Augmented Generation (RAG) / Grounding

This is the trickiest AI concept in the project. When a user asks LOOP a question (e.g. “What are users saying about onboarding?”), the AI must **never make up an answer from imagination**. Instead, the system: (1) turns the question into a vector (embedding), (2) searches stored feedback for the most similar items, (3) hands only those real items to Claude as context, and (4) Claude answers strictly from that evidence and cites which feedback items it used. This is called “grounding,” and it's what stops the AI from hallucinating fake customer quotes.

4. The Technology Stack, Explained

You don't get to pick your own stack — it's standardized so mentors can help everyone consistently. Here's what each piece is for, in plain terms:

Piece	Plain-English role
Next.js 14 + TypeScript	The framework — handles both the website (frontend) and the server logic (backend) in one codebase
Tailwind CSS	Styling shortcuts so you don't hand-write CSS files
PostgreSQL (Neon/Supabase)	The database that stores everything — companies, users, feedback, themes
Prisma	Translates your TypeScript code into database queries safely, with auto-generated types
NextAuth (Auth.js)	Handles login, logout, sessions, and “who is this user”
Anthropic Claude API	The AI brain — classifies feedback, answers questions, writes reports
pgvector / embeddings	Powers semantic search so “Ask LOOP” can find relevant feedback
Recharts	Draws the charts on your dashboard
Zod	Checks that data coming into your API is actually valid before you trust it
Vercel	Where you deploy the finished app so it has a public URL

Note: Java Full-Stack interns have an approved alternative — Spring Boot 3 + Java 17 + Spring Security + JPA/Hibernate + PostgreSQL on the backend, React (Vite) on the frontend. All requirements and grading stay identical either way — confirm with your mentor before Day 2.

5. How the Pieces Talk to Each Other (Architecture)

LOOP uses a simple three-layer flow. Memorize this — it's the backbone of every feature you build:

- **1. Browser** — the React UI. It never talks to the database or to Claude directly.
- **2. API layer (route handlers)** — checks who the user is, checks their role, and scopes every request to their workspace.
- **3. Database + Claude API** — Prisma reads/writes PostgreSQL; for AI features, your server builds a prompt, calls Claude, and returns clean JSON back to the browser.

Non-negotiable security rule

Your Anthropic API key must stay on the server, never in browser code or committed to Git. And again: every query touching tenant data must be filtered by workspaceId — no exceptions.

6. The Data Model (What Gets Stored)

This is the minimum database schema you must implement (you can add more fields, not remove these):

Table	What it stores
Workspace	One row per company/tenant
User	Login info + role (ADMIN/ANALYST/VIEWER), linked to a Workspace
Feedback	The actual feedback text, channel, sentiment, status, linked to a Workspace
Theme	A named topic/cluster (e.g. "Onboarding complaints")
FeedbackTheme	Join table linking feedback items to the themes they belong to, with a confidence score
Embedding	The vector representation of a feedback item, used for semantic search
Report	A saved Voice-of-Customer report, generated for a time period

Seed data is mandatory — ship a script that creates one demo workspace, one user per role, at least 120 realistic feedback items across several channels, and a handful of themes. A demo with 3 rows of data looks unfinished and will cost you marks.

7. The Features You Must Build

A. Core application features

Feature	In one sentence
Auth & Workspaces	Sign up creates a workspace; the creator becomes Admin; sessions persist; data is scoped per workspace
Role-Based Access	Admins manage members; Analysts manage feedback; Viewers are read-only — enforced server-side
Feedback Ingestion	Add feedback one at a time, via CSV bulk upload, or via a simulated “channel” button
Feedback Inbox	Paginated, searchable, filterable list with a NEW → REVIEWED → ACTIONED status workflow
Analytics Dashboard	At least 3 charts (volume over time, sentiment breakdown, top themes) + stat cards

B. AI features (the heaviest-weighted part of your grade)

AI Feature	What it does
AI1 — Auto-classification	Every new feedback item is sent to Claude and comes back tagged with sentiment, a score, theme(s),
AI2 — Theme clustering & trends	Similar feedback is grouped into named themes; a trends view shows which themes are growing and f
AI3 — Ask LOOP (grounded Q&A)	A chat box where anyone can ask plain-English questions; the system retrieves the most relevant real
AI4 — Voice-of-Customer report	One click generates a shareable digest: top themes, sentiment shifts, real quotes, and recommended

Implementation tip from the brief: for classification, ask Claude to return strict JSON and validate it with Zod before saving. For the report, pre-calculate the real stats in your own code first, then ask Claude to only write the narrative around those numbers — this stops the AI from inventing figures.

8. What's In Scope vs. Out of Scope

Build this (required)

- Multi-tenant workspaces with 3 roles
- Feedback ingestion: manual entry, CSV upload, and a simulated channel
- A feedback inbox with search, filters, pagination, status workflow
- An analytics dashboard with 3+ real charts
- All 4 AI features
- A public deployment, README, and demo video

Do NOT build this (explicitly excluded)

- Real third-party integrations (live Zendesk/App Store/Twitter pulls) — simulate with seed data instead
- Billing, payments, or subscription tiers
- Native mobile apps
- Real-time collaboration (websockets, live cursors)
- Email/SMS delivery infrastructure

Building excluded items doesn't earn extra credit — it actively lowers your score because it means time taken away from the required scope.

9. The 4-Week Build Plan

Twenty working days, broken into four weekly sprints. Each week ends with something demoable — don't move on until that week's deliverable actually works.

Week	Focus	You should be able to demo...
Week 1	Foundation & Data Layer	Sign up, log in, and add/view feedback on a live deployed URL
Week 2	Core Application	Bulk CSV import, a working filterable inbox, and a dashboard shell
Week 3	AI Integration	Auto-classification, theme trends, and Ask LOOP all working on real data
Week 4	Production Polish	VoC report, polished UI states, README, and a recorded demo video

If you fall behind: protect Week 3 (AI features) and the deployment — they carry the most marks. Cut polish before you cut core functionality, and flag blockers to your mentor early.

10. Milestones & How You're Graded

Milestone	Due	Points	Full marks means...
M1 Foundation	End Wk 1	10	Live URL, auth, RBAC, workspaces, basic CRUD all work
M2 Core App	End Wk 2	15	Bulk + single ingestion, fast paginated inbox, real dashboard
M3 AI Features	End Wk 3	15	Classification, trends, Ask LOOP all working with grounded answers
M4 Production	End Wk 4	10	VoC report, polished states, responsive UI, clean README + demo

These 50 “Project” marks sit inside Zidio's overall 100-point rubric: **Project 50** + **Submission 20** (covered by this brief) + **Attendance 20** + **Test 10** (assessed separately by the program).

Submission quality (20 marks)

Criterion	Marks	What earns full marks
Live deployment	5	Public URL, seeded, a grader can log in and explore
README & docs	5	Clear setup steps, architecture overview, screenshots
Code quality & Git	5	Typed, organized code; meaningful commit history; no leaked secrets
Demo video	5	3–5 minute walkthrough showing every feature working

The quality bar mentors are looking for

Average: features are present but rough — broken edge cases, ugly empty states, AI that sometimes invents data.

Top score: feels like a real product — fast, properly isolated per tenant, AI answers are grounded and useful, and you could genuinely show the demo to a paying customer.

11. What You Must Hand In

- GitHub repository link (public, or private with mentor access)
- Live Vercel deployment URL with seeded data and login credentials for each role
- A README.md: project summary, tech stack, local setup steps, env variables, seed commands, architecture summary, screenshots
- A 3–5 minute demo video (unlisted YouTube/Drive link) walking through every feature
- The completed submission form for your cohort
- A 1–2 minute self-feedback video about the internship

Reminder: your README must list one working login per role (Admin/Analyst/Viewer) on the seeded demo workspace, so graders can verify RBAC without creating their own accounts. Use throwaway demo passwords — never a real one you reuse elsewhere.

12. How to Actually Start (Day 1)

Prerequisites: Node.js 18+, Git, a free PostgreSQL database (Neon or Supabase), an Anthropic API key, and a Vercel account linked to GitHub.

Rough shape of the first-run setup:

- Scaffold the app: `npx create-next-app@latest loop --typescript --tailwind --app`
- Install dependencies: Prisma, NextAuth, Zod, Recharts, the Anthropic SDK
- Add environment variables (never commit them): `DATABASE_URL`, `NEXTAUTH_SECRET`, `ANTHROPIC_API_KEY`
- Run your first migration and seed script: `npx prisma migrate dev` then `npm run seed`
- Run locally, then deploy immediately with `vercel` — get a working “hello world” live on Day 1 so deployment is never a surprise later

13. How You're Expected to Work

- **TypeScript everywhere** — avoid using `any`; let types document your data
- **Validate everything server-side** with Zod — never trust data coming from the browser
- **Keep logic out of UI components** — business logic belongs in API routes / service files
- **Commit early and often** with meaningful messages — your git history should tell the story of the four weeks, not end in one giant “final” commit
- **Never commit secrets** — `.env` files and API keys must never reach GitHub
- **Ask for help by showing your work** — “my CSV import throws this error, here's the code and what I checked” gets a fast answer; “it doesn't work” does not

Originality rules

Using AI coding assistants is fine and expected — but you must understand and be able to explain every line you submit. You may reference tutorials, but the LOOP feature set, data model, and AI logic must be your own implementation. Copying another intern's repo or a purchased project results in disqualification, and mentors may ask you to explain or modify any part of your code live at final review.

14. Quick Glossary

Term	Meaning
Multi-tenant	One app serving many isolated customers; each tenant's data stays private
RBAC	Role-Based Access Control — permissions determined by a user's role
Tenant isolation	Guaranteeing one customer can never read/write another customer's data
Embedding	A numeric vector representing text meaning, used for similarity search
RAG	Retrieval-Augmented Generation — fetch relevant real data first, then let the AI answer from it
Grounding	Forcing the AI to answer only from provided data, never from imagination
VoC	Voice of Customer — the synthesized view of what customers are saying
Seed data	Sample records loaded into the DB so the app is demoable immediately

This explainer is a companion summary based on the Zidio Project LOOP brief (v1.0). Always defer to the original document and your mentor for anything this guide doesn't cover.